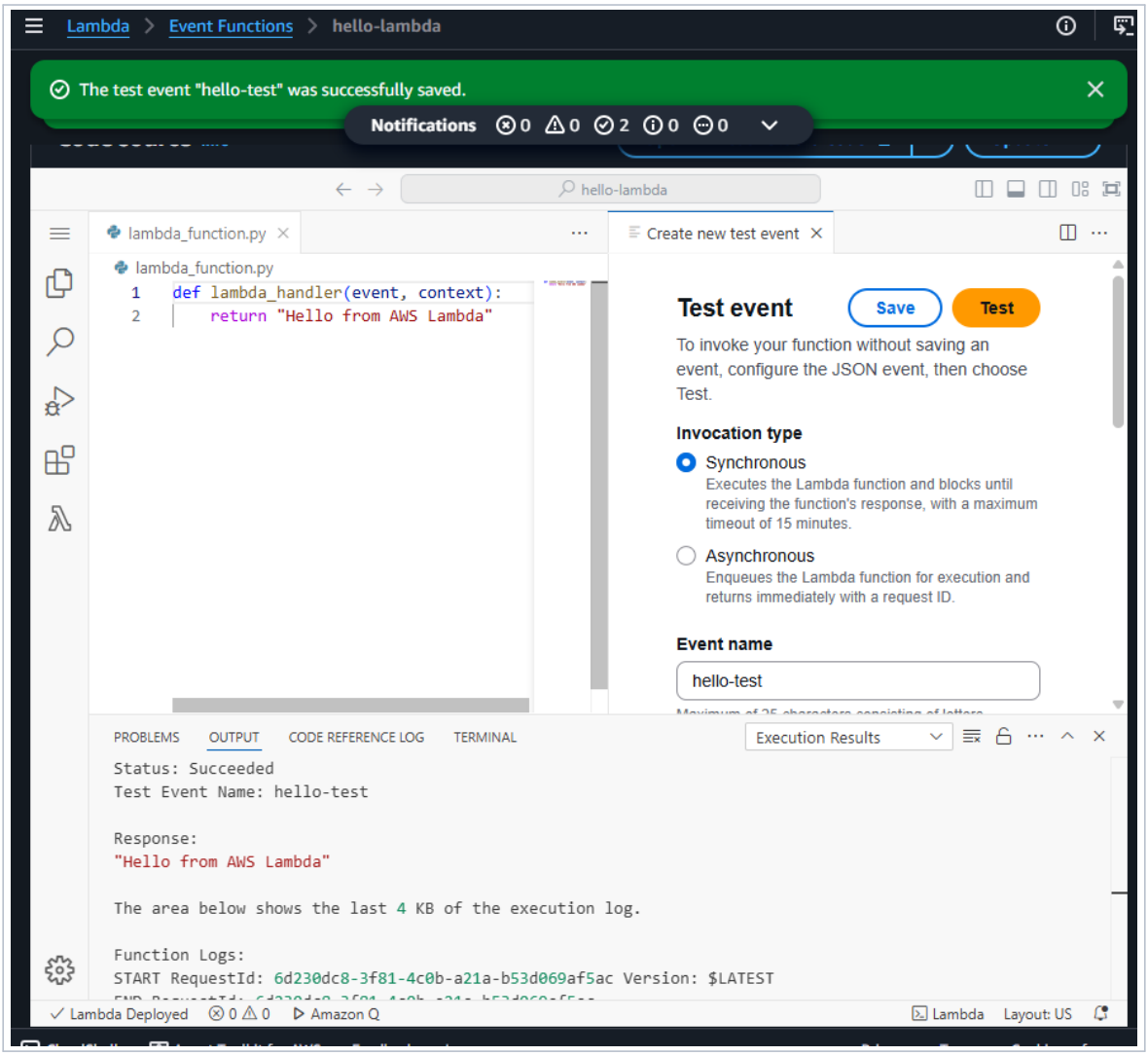
# AWS Lambda Lab Exercises

Submitted by Hrishinandan P

## Task 1: Create Your First Lambda Function

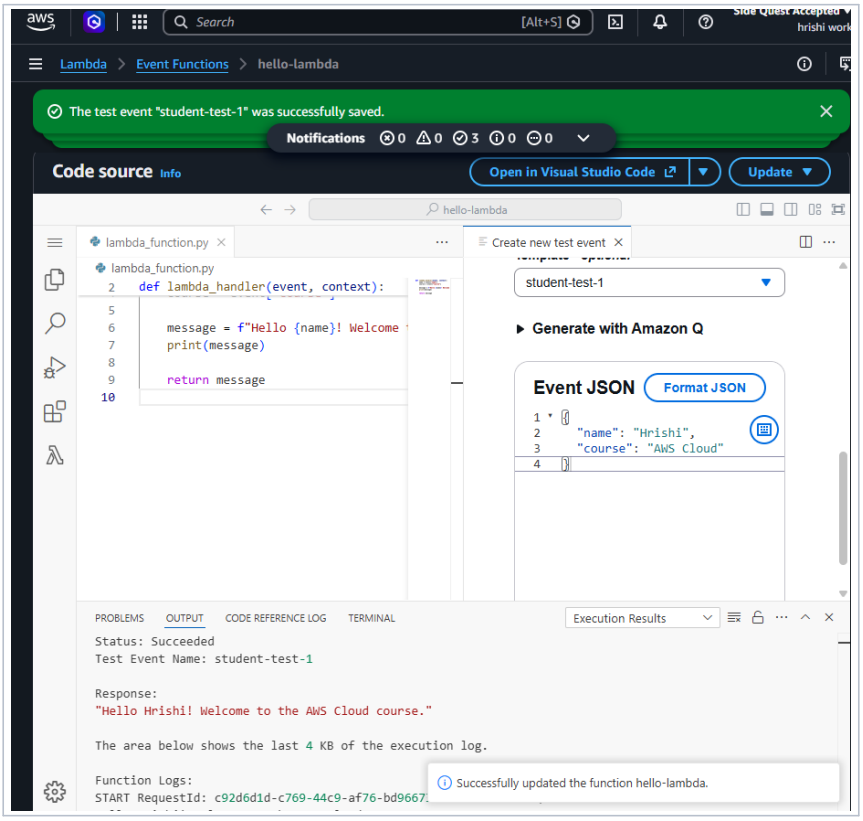
- Created a Python Lambda function **hello-lambda** that returns a greeting message.
- Deployed it and tested with the **hello-test** event; the execution succeeded with the expected output.



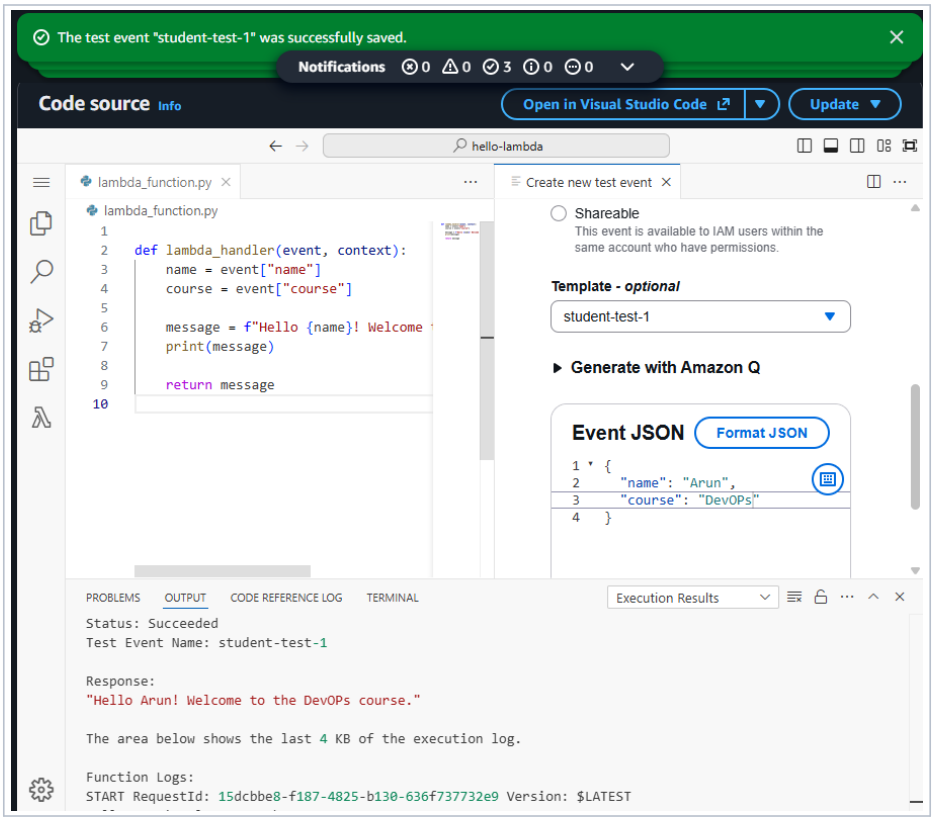

*Code, test event and successful output*

# Task 2: Read Event JSON

- Created test events with **name** and **course** and read them from the **event** object in the code.
- Tested with two different JSON inputs; each returned the correct personalized greeting.



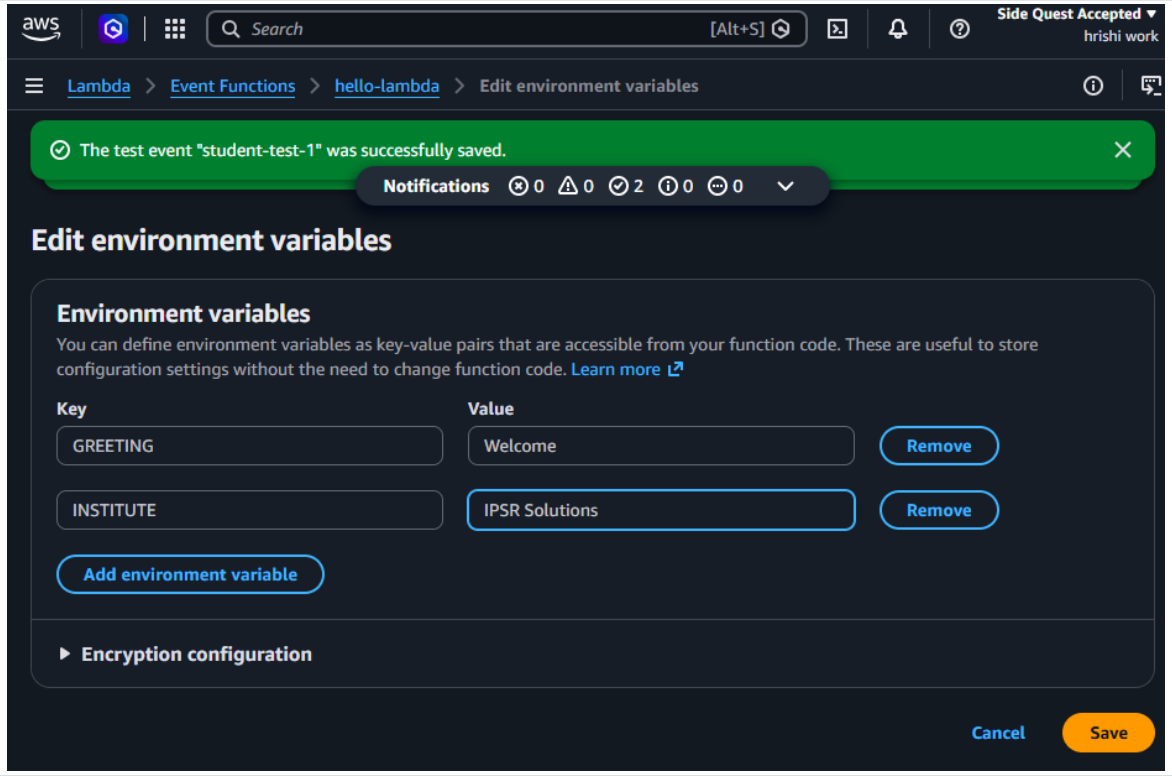

*Test 1: name = Hrishi, course = AWS Cloud*



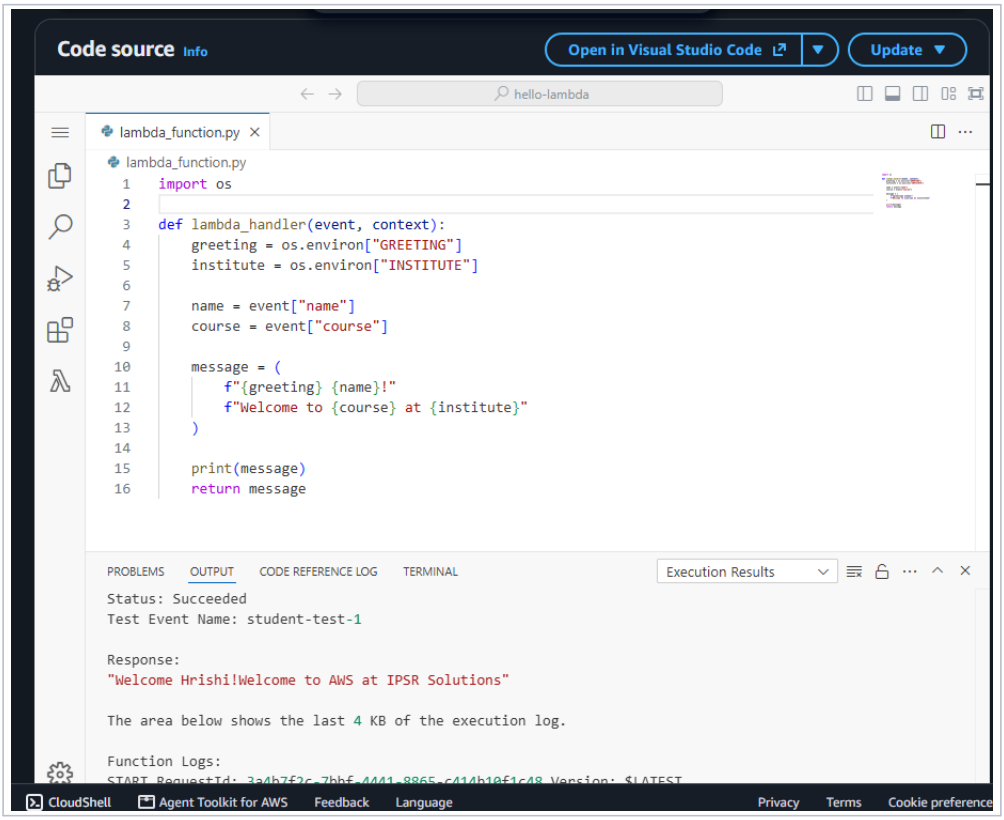

*Test 2: name = Arun, course = DevOps*

# Task 3: Use Environment Variables

- Created environment variables **GREETING** and **INSTITUTE**.
- Read them with Python's **os** module and combined them with the event values; the test succeeded.



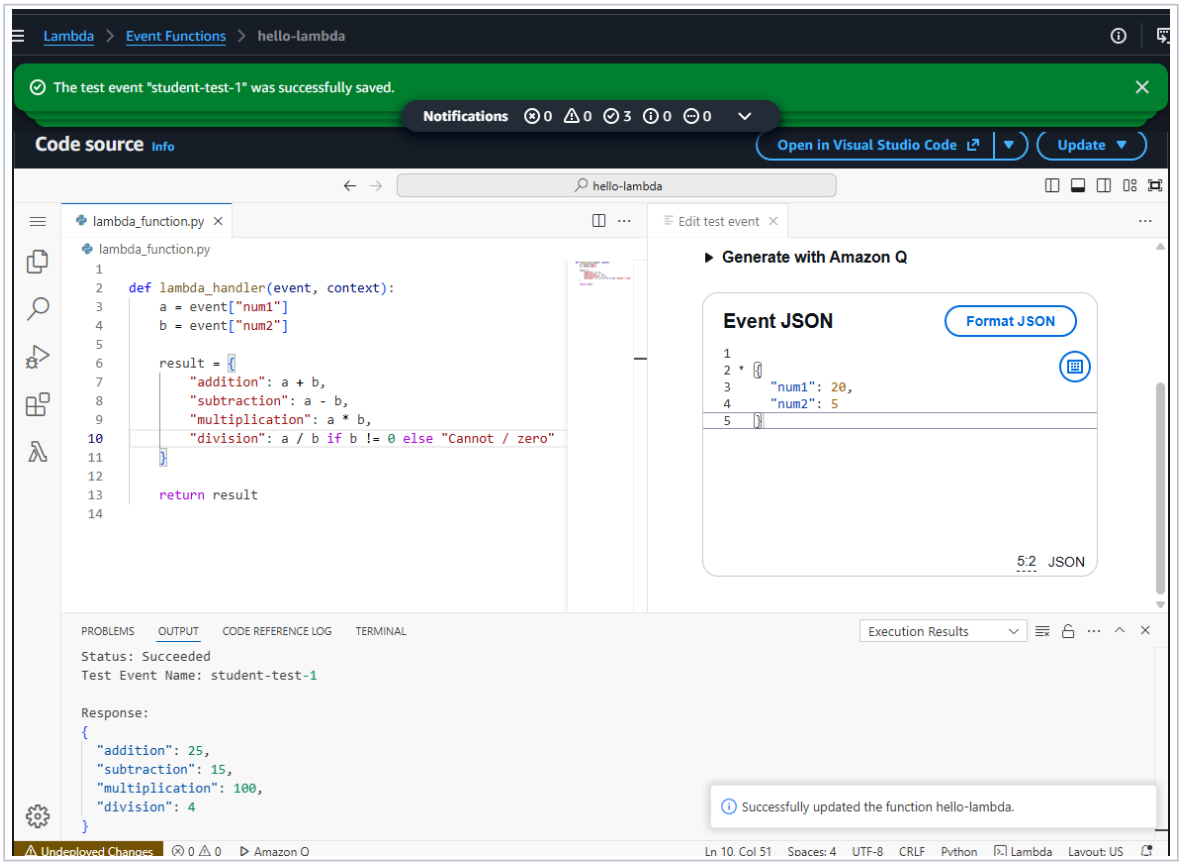

*Environment variables*




*Code and output*

# Task 4: Calculator

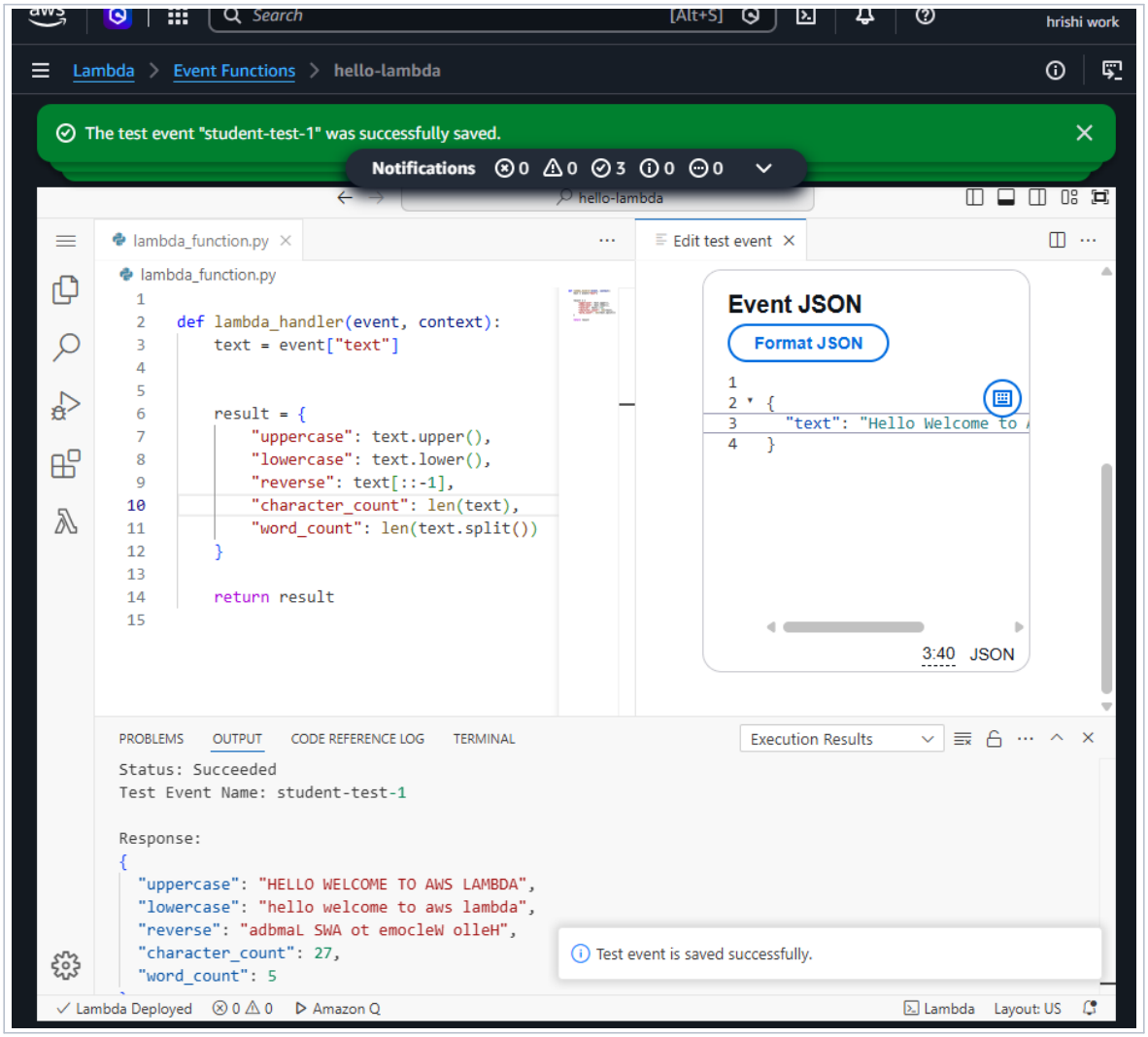
- Created a function that accepts two numbers (**num1**, **num2**) in the event JSON.
- Returns addition, subtraction, multiplication and division as a JSON response.




*Code, test event (20, 5) and JSON output*

# Task 5: String Operations

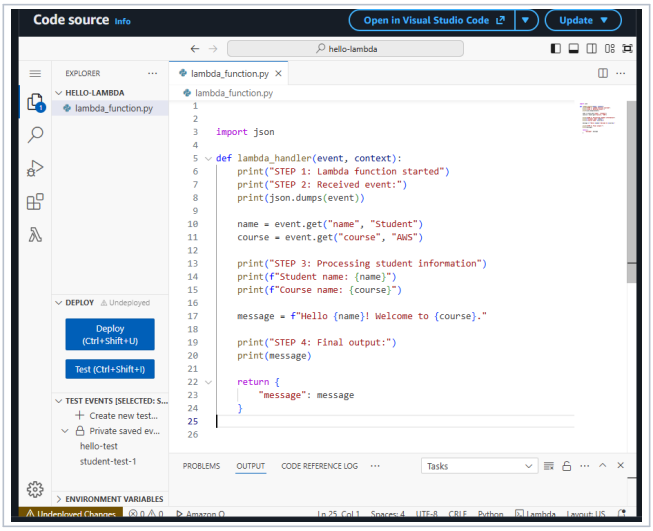
- Created a function that accepts a string (**text**) in the event JSON.
- Returns uppercase, lowercase, reverse, character count and word count as JSON.



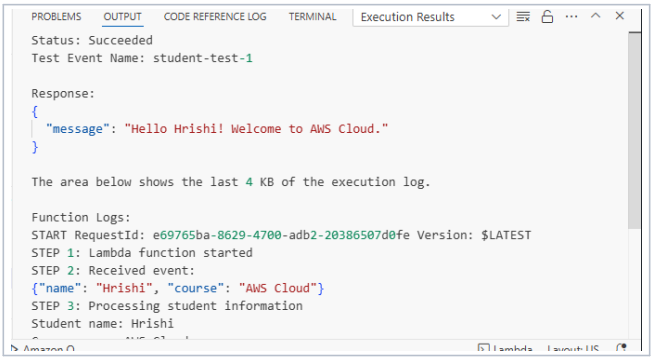

*Code and JSON output*

# Task 6: CloudWatch Logs

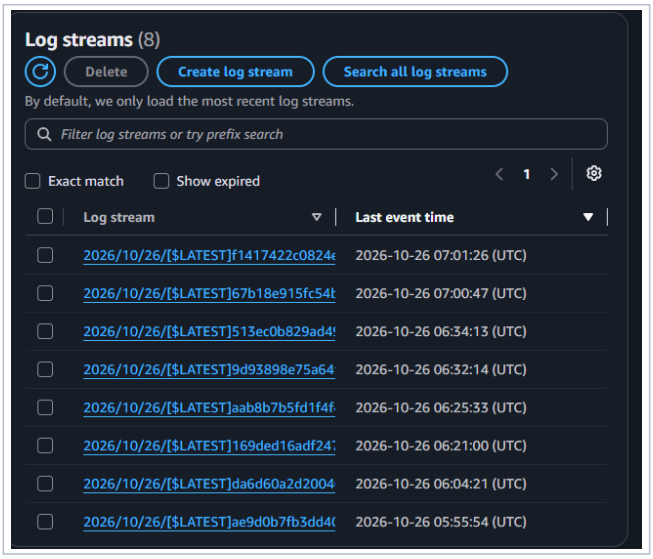
- Added print statements to log the received event, each processing step and the final output.
- Ran the function and verified the same log entries in CloudWatch Logs.



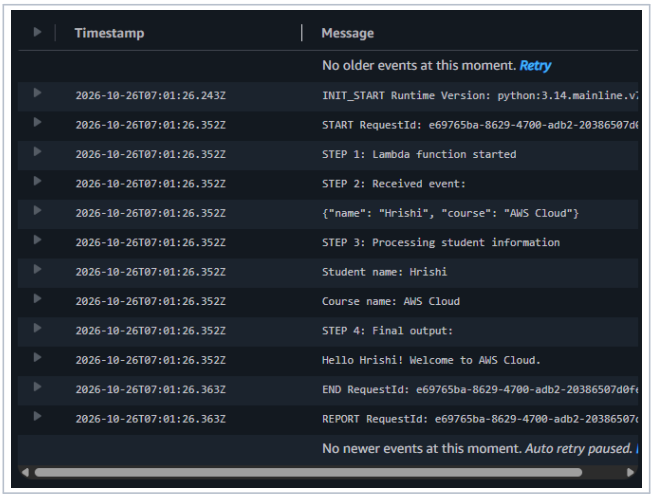

Code with print statements




Execution result and function logs



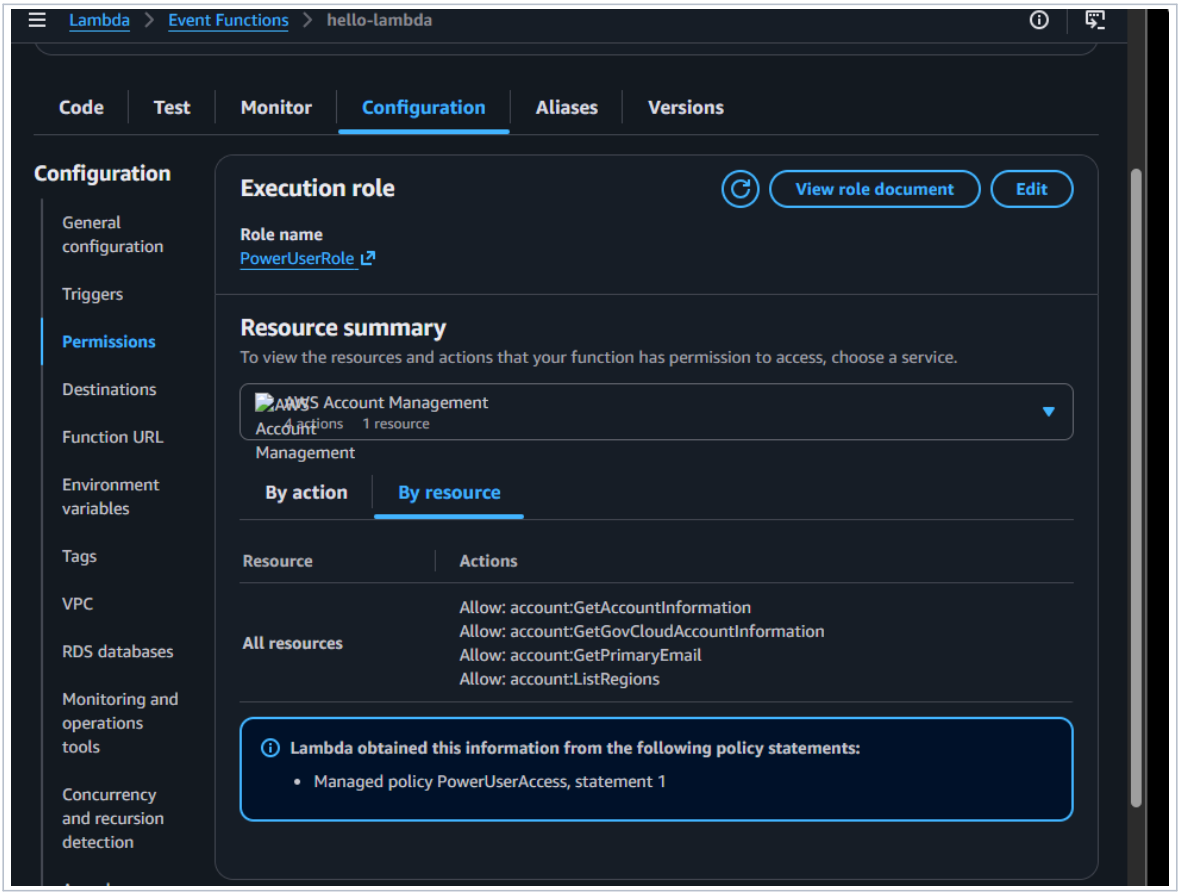

CloudWatch log streams




CloudWatch log events

# Task 7: IAM Execution Role

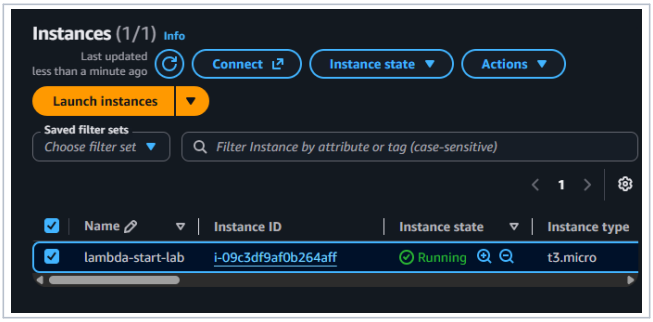
- Opened the execution role **PowerUserRole** from the Configuration tab; the attached policy is **PowerUserAccess**.
- The role is required because Lambda assumes it at runtime to get permission to use AWS services (such as CloudWatch and EC2).




*Configuration > Permissions > Execution role*

# Task 8: Start an EC2 Instance

- Created an EC2 instance and added an IAM policy allowing **ec2:StartInstances** to the Lambda execution role.
- Passed the instance ID in the test event and started the instance using boto3; the state changed to Running.




*EC2 instance created*




*IAM policy: ec2:StartInstances*

lambda_function.py

```python
import boto3

def lambda_handler(event, context):
    instance_id = event["instanse_id"]


    ec2 = boto3.cilent("ec2")

    response = ec2.start_instances(
        InstanceIds=[instanse_id]
    )

    return {
        "message": "EC2 start request submitted successfully",
        "instance_id": instance_id,
        "starting_state": response["StartingInstances"][0]
            ["CurrentState"]["Name"],
        "requested_state": response["StartingInstances"][0]
            ["CurrentState"]["Name"]
    }
```

*Lambda code (boto3)*

```
Status: Succeeded
Test Event Name: start-ec2-test

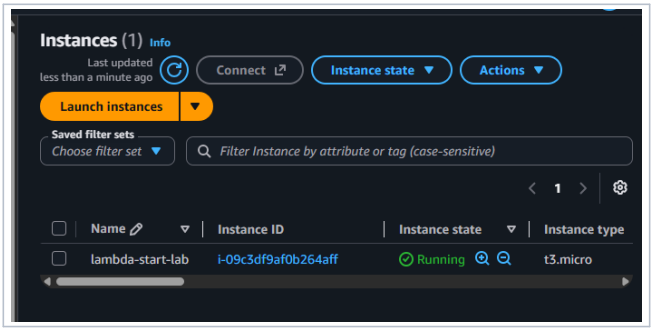
Response:
{
  "message": "EC2 start request submitted successfully",
  "instance_id": "i-09c3df9af0b264aff",
  "state": "running"
}

The area below shows the last 4 KB of the execution log.

Function Logs:
START RequestId: 527687cb-c884-465f-bb54-87c629f3e480 Version: $LATEST
END RequestId: 527687cb-c884-465f-bb54-87c629f3e480
REPORT RequestId: 527687cb-c884-465f-bb54-87c629f3e480  Duration: 1138.95 ms
Billed Duration: 1139 ms    Memory Size: 128 MB Max Memory Used: 102 MB

Request ID: 527687cb-c884-465f-bb54-87c629f3e480
```

*Test output*



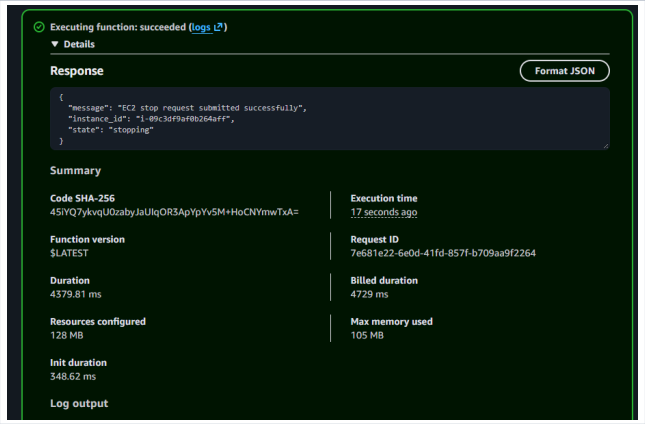

*Instance state: Running*

# Task 9: Stop an EC2 Instance

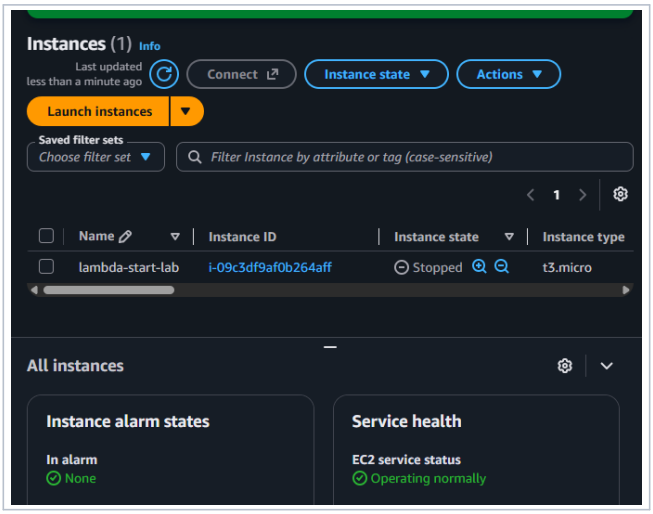
- Extended the IAM policy with **ec2:StopInstances** and modified the function to stop the instance from the event JSON.
- Ran the test; the function succeeded and the instance state changed to Stopped.




*IAM policy: StartInstances and StopInstances*




*Test output*




*Instance state: Stopped*

# Task 10: EC2 Management Mini Project

- One function accepts **action** (start or stop) and **instance_id**, and starts or stops the instance accordingly.
- Invalid actions return a 400 error message, and all operations are logged to CloudWatch.

lambda_function.py

```python
logger = logging.getLogger()
logger.setLevel(logging.INFO)

ec2 = boto3.client("ec2")

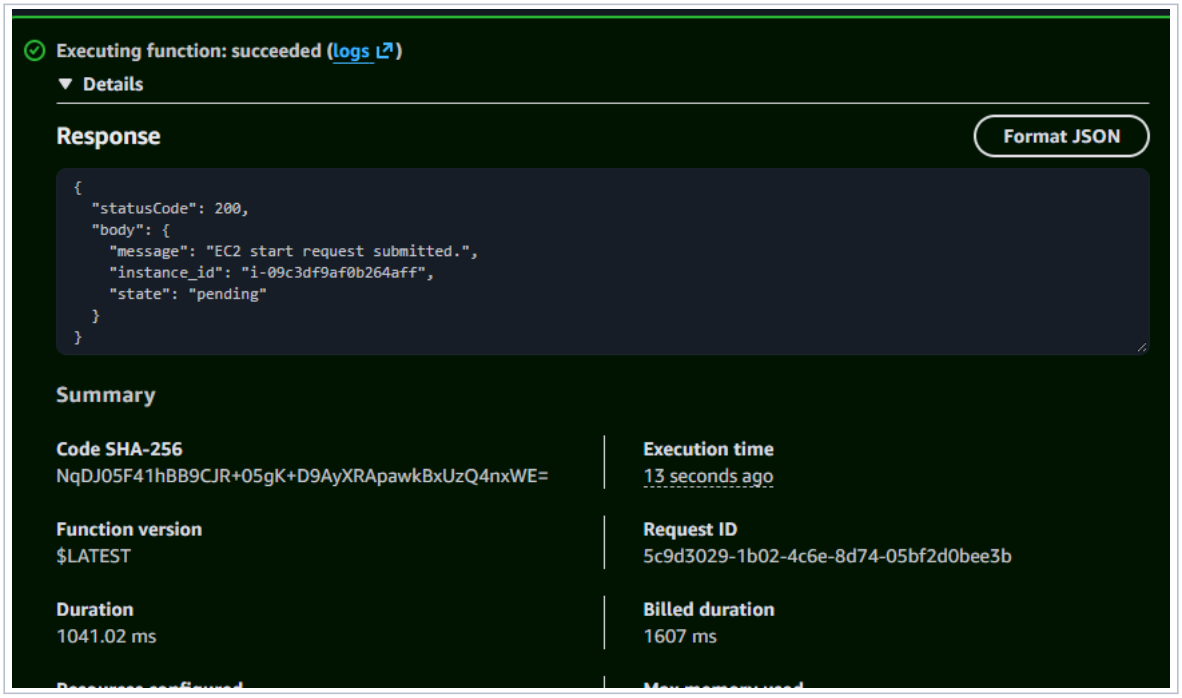

def lambda_handler(event, context):
    logger.info("Received event: %s", event)

    action = event.get("action", "").strip().lower()
    instance_id = event.get("instance_id", "").strip()

    if action not in ("start", "stop"):
        logger.warning("Invalid action received: %s", action)
        return {
            "statusCode": 400,
            "body": {
                "error": "Invalid action. Use 'start' or 'stop'."
            }
        }

    if not instance_id:
        logger.warning("Missing instance_id")
        return {
            "statusCode": 400,
            "body": {
```

*Lambda code with validation and logging*




*Test output (action = start)*